SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I __Mathivadhani.K(192311424)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Mathivadhani.K

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

1. Online Banking – Equivalence Partitioning, BVA and Random Testing

A. Problem analysis

The online banking system frequently fails during fund transfers, particularly at minimum and maximum transfer limits. This indicates inadequate boundary testing, improper input validation, or incorrect handling of transaction limits.

B. Apply black-box testing techniques

Assume the permitted transfer amount is ₹100 to ₹50,000.

Equivalence Class Partitioning (ECP): Divide inputs into valid and invalid classes.

Equivalence class	Test input	Expected result
Below minimum	₹99	Reject
Minimum boundary	₹100	Accept
Valid range	₹5,000	Accept
Maximum boundary	₹50,000	Accept
Above maximum	₹50,001	Reject
Zero or negative	₹0, −₹100	Reject

Boundary Value Analysis (BVA): Test values immediately below, at, and above each boundary.

Boundary	Test values
Minimum = ₹100	₹99, ₹100, ₹101
Maximum = ₹50,000	₹49,999, ₹50,000, ₹50,001

C. Evaluate random testing

Random testing may discover unexpected inputs, but it can miss boundary defects because the probability of selecting exact boundary values is low. Therefore, it should be retained as supplementary testing, not replaced as the primary strategy.

D. Improved testing strategy

1. Apply ECP to reduce redundant test cases.
2. Apply BVA to detect off-by-one and limit-validation errors.
3. Test insufficient balance, invalid beneficiary, duplicate requests, and network failures.
4. Verify that failed transfers do not deduct money and successful transfers do not cause duplicate debits.
5. Use automated regression testing after every correction.

Conclusion: Combining ECP, BVA, transaction-integrity tests, and supplementary random testing improves defect detection while reducing unnecessary test cases.

2. Healthcare Application – Control Flow Graph and Coverage

A. Problem analysis

Critical defects found after deployment indicate inadequate test coverage. The team may have checked individual statements without testing alternative decisions, exception paths, or combinations of conditions.

B. Conceptual Control Flow Graph (CFG)

Consider a healthcare module that validates a patient's details before processing a prescription.
mermaid

```
flowchart TD
    A([Start]) --> B[Read patient details]
    B --> C{Patient valid?}
    C -- No --> D[Display error]
    D --> I([End])
    C -- Yes --> E{Prescription valid?}
    E -- No --> F[Reject prescription]
    F --> I
    E -- Yes --> G[Save prescription]
    G --> H[Update medical record]
    H --> I
```

C. Identify missing paths

Test case	Patient valid?	Prescription valid?	Expected path
TC1	No	Not evaluated	Error message
TC2	Yes	No	Reject prescription
TC3	Yes	Yes	Save and update record

These three cases exercise the principal decision outcomes. Additional tests should cover database failures, missing fields, invalid patient identifiers, and recovery from interrupted updates.

D. Evaluate coverage criteria

- Statement coverage: Every executable statement is executed at least once.
- Branch coverage: Every decision outcome (True and False) is executed.
- Path coverage: Every feasible execution path is tested.

Statement coverage alone is insufficient because it may not exercise all decision outcomes. Complete path coverage can also become impractical when loops and many conditions exist.

E. Improved strategy

Use CFG-based test design, achieve 100% statement and branch coverage for the relevant code, test feasible independent paths, and add exception, integration, and regression tests. Healthcare data updates should also be checked for consistency and recovery.

Conclusion: CFG analysis exposes untested execution paths, while stronger coverage criteria and risk-based testing reduce the chance of critical defects reaching patients.

3. Retail E-Commerce – ECP, BVA and Decision Table Testing

A. Problem analysis

The system calculates the final amount using price, quantity, and discount. Testing each field separately may miss errors caused by interactions, such as a discount exceeding the price or a zero quantity combined with a discount.

B. Apply ECP and BVA

Assume unit price must be positive, quantity must be a positive integer, and discount must be between 0% and 50%.

Input	Valid class	Invalid classes	Important boundary tests
Price	Price > ₹0	Zero or negative	₹0, ₹0.01, ₹0.02
Quantity	Integer ≥ 1	Zero, negative, fractional	0, 1, 2
Discount	0–50%	Below 0%, above 50%	−1%, 0%, 1%, 49%, 50%, 51%

C. Apply Decision Table Testing

Rule	Valid price	Valid quantity	Valid discount	Expected result
R1	Yes	Yes	Yes	Calculate final amount
R2	No	Yes	Yes	Reject price
R3	Yes	No	Yes	Reject quantity
R4	Yes	Yes	No	Reject discount
R5	Yes	Yes	Yes, maximum	Accept and apply maximum discount

For each valid rule, test representative values and important combinations. For example, price ₹1,000, quantity 2, and discount 10% should produce ₹1,800 before taxes or shipping.

D. Revised strategy

- Use ECP to partition valid and invalid inputs.
- Use BVA for zero, minimum, maximum, and just-outside-limit values.
- Use decision tables for combinations of business rules.
- Use pairwise testing when many input parameters create a large combination space.
- Verify rounding, currency precision, tax calculation, and error messages.

Conclusion: ECP and BVA detect individual input defects, while decision tables and combination testing detect interaction defects, improving test completeness without exhaustively testing every possible value.

4. Complex Nested Conditions and Loops – Path Testing

A. Problem analysis

The software module contains nested if-else statements and loops, but only partial path coverage has been achieved. This may leave certain combinations of conditions, loop exits, and error-handling paths untested.

B. Control Flow Graph and independent paths

Consider the following logic:

flowchart TD

A([Start]) --> B{Input valid?}

B -- No --> C[Report error]

C --> H([End])

B -- Yes --> D{Items remain?}

D -- No --> H

D -- Yes --> E{Condition satisfied?}

E -- Yes --> F[Process item]

E -- No --> G[Skip item]

F --> D

G --> D

The graph contains decisions for input validation, loop continuation, and item processing.

C. Cyclomatic complexity

Cyclomatic complexity estimates the number of linearly independent paths.

$V(G)=E-N+2P$

$V(G)=E-N+2P$

Where

E-E is the number of edges,

N-N is the number of nodes, and

P-P is the number of connected components.

For a single connected graph,

P=1

P=1, so

$V(G)=E-N+2$

$V(G)=E-N+2$.

It can also be calculated as the number of decision nodes plus one. In the example, there are three decision nodes, giving

V(G)=4

$V(G)$=4.

D. Improved path testing strategy

Test	Scenario	Expected result
TC1	Invalid input	Error and termination
TC2	Valid input, zero items	Loop skipped
TC3	One item satisfying condition	Item processed
TC4	One item not satisfying condition	Item skipped
TC5	Multiple items	Loop repeats and terminates

Test loops for zero, one, and multiple iterations, including exit conditions and invalid data. For complex loops, use bounded iteration testing and test nested-loop combinations based on risk.

Conclusion: CFG construction and basis path testing systematically identify independent execution paths. Cyclomatic complexity helps estimate the minimum basis set of paths, improving structural coverage and defect detection. Exhaustive path coverage may be infeasible for loops with unbounded iterations.

5. Financial Application – Hybrid Black-Box and White-Box Testing

A. Problem analysis

Although both testing methods are used, production defects continue to occur. Possible reasons include inadequate boundary tests, incomplete branch coverage, incorrect requirements, and failure to test interactions between financial calculations and internal control flow.

B. Compare the techniques

Black-box testing	White-box testing
Tests external behaviour against requirements	Tests internal code structure and execution
Uses ECP, BVA and decision tables	Uses CFG, branch and basis path testing
Detects incorrect outputs and invalid-input handling	Detects unexecuted branches and logic defects
Does not require code knowledge	Requires access to code or its structure

C. Proposed hybrid test strategy

Assume a financial transfer allows amounts from ₹100 to ₹50,000.

Black-box tests:

- Test ₹99, ₹100, ₹101, ₹49,999, ₹50,000 and ₹50,001.
- Test insufficient balance, invalid accounts, and zero or negative amounts.
- Verify the transaction amount, fees, balance, and rejection messages.

White-box tests:

- Construct a CFG for validation, balance checking, transfer execution, and error handling.
- Test both outcomes of every decision.
- Use basis path testing to exercise independent paths.
- Test exception handling, database rollback, and transaction failure paths.

D. Test adequacy criteria

Measure statement and branch coverage, relevant condition or path coverage, and requirements coverage. No single percentage guarantees correctness; coverage must be combined with meaningful assertions and financial invariants.

For example:

Balance after transfer=Balance before transfer−Amount

Balance after transfer=Balance before transfer−Amount

This should hold for successful transfers, with the appropriate treatment of fees.

Conclusion

A hybrid approach combines external functional correctness with internal structural coverage. Automated regression testing, transaction-integrity checks, code reviews, and risk-based prioritization further reduce production defects.

6. HR Payroll – Bonus Errors at 5, 10 and 15 Years

A. Problem analysis

Employees with exactly 5, 10, or 15 years of service receive incorrect bonuses. This strongly suggests boundary errors, such as using > instead of >=, incorrect range definitions, or overlapping service brackets.

B. Apply Equivalence Class Partitioning

Assume the bonus category changes at 5, 10, and 15 years. The exact bonus amounts depend on the organization's policy.

Service period	Equivalence class
Below 5 years	Class 1
5 to below 10 years	Class 2
10 to below 15 years	Class 3
15 years and above	Class 4
Negative service	Invalid class

C. Apply Boundary Value Analysis

Boundary	Test values in years	Purpose
5 years	4.99, 5, 5.01	Verify first threshold
10 years	9.99, 10, 10.01	Verify second threshold
15 years	14.99, 15, 15.01	Verify third threshold

If service is recorded only in completed years, use 4, 5, 6 and similar integer values instead of fractional years.

D. Improved test design

- Test each service boundary with multiple performance ratings.
- Verify the bonus amount against the official payroll formula.
- Test minimum and maximum performance ratings.
- Check leap-year and service-date calculations where applicable.
- Compare calculated bonuses with independently verified expected results.

Exploratory testing may discover unexpected issues, but it cannot reliably guarantee that every critical threshold has been tested. ECP and BVA provide systematic, repeatable coverage of these boundaries.

Conclusion: ECP reduces redundant tests, while BVA directly targets off-by-one errors at 5, 10, and 15 years. Combining them with performance-rating tests improves payroll accuracy and prevents incorrect employee payments.

7. Laboratory Temperature Control – CFG and 100% MC/DC

A. Problem analysis

The system controls heaters, coolers, and alarms using nested conditions. An alarm failure during sensor malfunction indicates that certain Boolean-condition combinations or decision outcomes may not have been tested.

B. Conceptual control flow

Assume the system checks sensor validity, high temperature, and low temperature.

flowchart TD

A([Start]) --> B{Sensor valid?}

B -- No --> C[Activate alarm and safe state]

B -- Yes --> D{Temperature above limit?}

D -- Yes --> E[Turn cooler ON]

D -- No --> F{Temperature below limit?}

F -- Yes --> G[Turn heater ON]

F -- No --> H[Maintain normal state]

C --> I([End])

E --> I

G --> I

H --> I

C. Branch coverage versus MC/DC

- Branch coverage: Executes both True and False outcomes of every decision.
- Condition coverage: Makes every individual Boolean condition True and False.
- MC/DC: Demonstrates that each condition independently affects the outcome of its decision while other conditions are held appropriately constant.

For a safety-critical system, branch coverage alone may miss interactions between Boolean conditions.

D. MC/DC test design

Suppose the alarm decision is:

$A=\neg S \vee H$

$A=\neg S \vee H$

Where

S-*S* means sensor valid and

H-*H* means temperature above the safety limit.

Test	Sensor valid S	High temperature H	Alarm A
TC1	True	False	False
TC2	False	False	True
TC3	True	True	True

- TC1 versus TC2 demonstrates the independent effect of sensor validity.
- TC1 versus TC3 demonstrates the independent effect of high temperature.

E. Improved strategy and conclusion

Achieve 100% MC/DC for the relevant safety decisions, test sensor disconnection and implausible readings, and verify safe-state transitions through hardware-in-the-loop testing. Also test alarm activation, recovery, and failure handling.

MC/DC provides stronger evidence that each condition independently influences safety decisions, helping reveal logic defects that ordinary branch coverage may miss.

8. Travel Booking Portal – Decision Table Testing

A. Problem analysis

The travel portal allows multiple promotional codes, and some combinations produce a negative total amount. Testing each promo code independently is insufficient because the defect may arise from interactions between discounts, user tiers, and the original price.

B. Identify input conditions

Assume the system supports these illustrative rules:

- Standard promo: 10% discount.
- Premium promo: 20% discount.
- Premium users receive an additional 5% discount.
- Discounts cannot make the final amount negative.
- Only one promotional code may be applied per booking.

The actual rules must be confirmed against the business requirements.

C. Decision table

Rule	Valid price	Promo valid	Premium user	Multiple codes	Expected result
R1	Yes	Yes	No	No	Apply permitted discount
R2	Yes	Yes	Yes	No	Apply eligible tier benefit
R3	Yes	No	Either	No	Reject promo
R4	No	Either	Either	Either	Reject price
R5	Yes	Yes	Either	Yes	Reject or handle according to policy

D. Optimized test set

Test	Price	Promo combination	User tier	Expected result
TC1	₹1,000	Standard	Basic	Valid non-negative total
TC2	₹1,000	Premium	Premium	Apply eligible discount
TC3	₹1,000	Invalid code	Basic	Reject code
TC4	₹0	Standard	Basic	Reject if price must be positive
TC5	₹100	Multiple codes	Premium	Enforce combination policy
TC6	₹100	Maximum permitted discount	Premium	Total must not be negative

E. Conclusion

Decision table testing systematically covers combinations of conditions and actions. Equivalence partitioning can reduce redundant inputs, while boundary testing checks zero prices and maximum discounts. Together, these techniques detect discount-stacking errors and prevent invalid or negative totals.

9. Legacy Database Migration – Path Testing and Cyclomatic Complexity

A. Problem analysis

The migration tool achieves high statement coverage but still experiences infinite-loop crashes. This occurs because executing every statement does not guarantee that loop termination conditions, exceptional paths, or combinations of loop conditions have been tested.

B. Example control flow

Consider a nested loop that validates records and their fields.

flowchart TD

A([Start]) --> B{Records remain?}

B -- No --> F[Finish migration]

B -- Yes --> C[Read record]

C --> D{Fields remain?}

D -- Yes --> E[Validate field]

E --> D

D -- No --> G[Save record]

G --> B

F --> H([End])

C. Why statement coverage is insufficient

- A loop body may execute once without testing its exit condition adequately.
- Incorrect loop-variable updates may cause infinite repetition.
- Empty input, one record, and multiple records may behave differently.
- Nested loops may terminate correctly for ordinary data but fail for malformed records.

D. Path and loop testing strategy

Test case	Input condition	Expected result
TC1	Zero records	Immediate termination
TC2	One record with zero fields	Record handled according to specification
TC3	One record with one field	Validation and termination
TC4	Multiple records and fields	All records processed exactly as expected

TC5	Malformed record	Controlled error and recovery
TC6	Large dataset	Completion within acceptable resource limits

Cyclomatic complexity is calculated as:

V(G)=E−N+2P

$V(G)=E-N+2P$

For a connected graph,

V(G)=E−N+2

$V(G)=E-N+2$. It estimates the number of independent basis paths that should be considered, but it does not directly measure loop termination or guarantee defect-free code.

E. Conclusion

Use basis path testing, zero/one/multiple-iteration loop tests, explicit termination checks, and timeouts. Statement coverage should be supplemented with branch coverage and data-integrity verification to reduce infinite-loop failures during migration.

10. Food Delivery App – State Transition Testing

A. Problem analysis

The food delivery tracker transitions through Ordered, Preparing, Out for Delivery, and Delivered. The app becomes stuck when the driver cancels during delivery. This indicates an untested state transition or an incorrectly handled exceptional event.

B. State transition model

stateDiagram-v2

[*] --> Ordered

Ordered --> Preparing: Restaurant accepts

Ordered --> Cancelled: Order cancelled

Preparing --> OutForDelivery: Driver collects

Preparing --> Cancelled: Cancellation permitted

OutForDelivery --> Delivered: Successful delivery

OutForDelivery --> Reassigned: Driver cancels

Reassigned --> OutForDelivery: New driver assigned

Reassigned --> Cancelled: No driver available

Delivered --> [*]

Cancelled --> [*]

C. State transition test cases

Test	Current state	Event	Expected next state
TC1	Ordered	Restaurant accepts	Preparing
TC2	Preparing	Driver collects order	Out for Delivery
TC3	Out for Delivery	Driver cancels	Reassigned
TC4	Reassigned	New driver assigned	Out for Delivery
TC5	Reassigned	No driver available	Cancelled
TC6	Delivered	Cancellation requested	Reject invalid transition

D. Improved testing strategy

- Test every valid transition at least once.
- Test invalid events in each relevant state.
- Verify notifications, refunds, delivery assignments, and tracking updates.
- Test repeated cancellation events and delayed network responses.
- Check that the system never remains indefinitely in an intermediate state.

E. Conclusion

ECP and BVA test input values, but they may not expose errors caused by the order of events or changes in system state. State Transition Testing directly validates event-driven workflows and ensures that cancellation, reassignment, and completion are handled correctly.

11. Cybersecurity Authentication – Hybrid Testing

A. Problem analysis

The authentication module checks password strength, OTP timing, and IP location, yet brute-force vulnerabilities remain. This may indicate missing rate limits, inadequate account lockout logic, incomplete authentication paths, or insufficient testing of repeated failures.

B. Black-box and white-box integration

Black-box techniques:

- Test valid and invalid passwords.
- Test correct, incorrect, expired, and reused OTPs.
- Test trusted and untrusted IP addresses.
- Test repeated login attempts, rate limiting, and account lockout.

White-box techniques:

- Construct a CFG for password validation, OTP verification, risk evaluation, and authentication success or failure.
- Identify independent paths through the authentication decisions.
- Verify that every failed attempt follows the appropriate logging and rate-limiting path.

C. Error Guessing test cases

Test	Scenario	Expected result
TC1	Incorrect password repeatedly	Attempts limited and monitored
TC2	Expired OTP	Authentication rejected
TC3	Correct password, incorrect OTP	Access denied
TC4	Valid credentials from suspicious IP	Additional verification or denial according to policy
TC5	OTP replay	Reuse rejected
TC6	Simultaneous login attempts	Rate limits remain effective

D. Improved hybrid strategy

Combine ECP and BVA for password and OTP inputs, error guessing for likely security weaknesses, and CFG-based path analysis for authentication decisions. Add automated security testing, audit-log verification, session management tests, and regression tests.

Rate limiting should be enforced consistently across relevant accounts and requests, with controls against distributed attempts. Sensitive responses should not reveal whether a username or password was correct unnecessarily.

E. Conclusion

Black-box testing checks observable security behaviour, while white-box testing reveals missing internal controls and untested paths. Their combination improves authentication robustness and helps prevent brute-force attacks.

12. E-Learning Platform – ECP for Subscription Dates and Types

A. Problem analysis

The platform provides Free, Basic, and Premium subscriptions. Errors occur when a user upgrades exactly on the expiration date. This suggests incorrect date comparisons, improper handling of inclusive and exclusive dates, or incorrect subscription-transition rules.

B. Apply Equivalence Class Partitioning

Divide subscription dates into valid and invalid classes.

Equivalence class	Test condition	Expected result
Before expiration	Upgrade before expiry	Apply upgrade policy
On expiration date	Upgrade exactly on expiry date	Follow defined expiry policy
After expiration	Upgrade after expiry	Apply renewal or reactivation rules
Invalid date	Missing or malformed date	Reject invalid input
Future start date	Subscription not yet active	Restrict access until activation

Subscription types should also be partitioned into Free, Basic, Premium, and invalid or unsupported types.

C. Boundary Value Analysis

Assume the subscription expires on 31 October 2026.

Test	Upgrade date	Purpose
TC1	30 October 2026	One day before expiry
TC2	31 October 2026	Exact expiration date
TC3	1 November 2026	One day after expiry

The expected outcome on the expiration date must be specified by the business rules. Tests should also cover the exact expiration timestamp and relevant time-zone differences.

D. Improved test strategy

- Test all subscription types and permitted upgrade paths.
- Verify access permissions before and after upgrading.
- Test payment success, payment failure, and repeated upgrade requests.
- Verify that expiry dates, billing dates, and course access remain consistent.
- Use regression testing after correcting date-comparison logic.

Conclusion: ECP reduces redundant tests by grouping equivalent date and subscription inputs, while BVA focuses on the critical expiration boundary. Together, they improve subscription accuracy and prevent unexpected access interruptions.

13. Smart-Home Lighting – Branch and Condition Coverage

A. Problem analysis

The lighting controller uses switch-case statements and Boolean expressions to adjust brightness based on ambient light. Relying only on statement coverage is risky because every statement may execute while certain decision outcomes or condition combinations remain untested.

B. Example control flow

Assume the controller uses the following logic:

- If the sensor is invalid, activate a safe fallback.
- Otherwise, if ambient light is low, increase brightness.
- If ambient light is high, reduce brightness.
- For normal light, maintain the current level.

```
flowchart TD
    A([Start]) --> B{Sensor valid?}
    B -- No --> C[Apply safe fallback]
    B -- Yes --> D{Ambient light low?}
    D -- Yes --> E[Increase brightness]
    D -- No --> F{Ambient light high?}
    F -- Yes --> G[Reduce brightness]
    F -- No --> H[Maintain brightness]
    C --> I([End])
    E --> I
    G --> I
    H --> I
```

C. Branch and condition coverage

- Statement coverage: Executes each statement at least once.
- Branch coverage: Exercises every decision outcome, both True and False.
- Condition coverage: Makes each individual Boolean condition True and False.

For example, testing only low ambient light may execute the brightness-increase statement but leave the reduction and safe-fallback branches untested.

D. Improved white-box test strategy

Test	Sensor status	Ambient condition	Expected action
TC1	Invalid	Any	Apply safe fallback
TC2	Valid	Low	Increase brightness
TC3	Valid	High	Reduce brightness
TC4	Valid	Normal	Maintain brightness

Additionally, test every switch-case option, default cases, threshold values, sensor disconnection, and actuator failures. Where decisions contain multiple Boolean conditions, apply MC/DC when required by the risk assessment.

E. Conclusion

Branch and condition coverage provide stronger evidence than statement coverage alone. CFG-based testing combined with hardware-in-the-loop tests helps detect missing logic paths and prevents software decisions from causing incorrect hardware behaviour.

14. Logistics System – Robust Boundary Value Analysis

A. Problem analysis

The shipping-cost system calculates charges based on weight, destination zone, and delivery speed. Errors involving zero or negative weight indicate inadequate input validation and insufficient testing of invalid boundary values.

B. Apply Robust BVA

Assume the permitted shipping weight is greater than 0 kg and no more than 100 kg.

Test case	Weight	Expected result

TC1	−0.01 kg	Reject
TC2	0 kg	Reject
TC3	0.01 kg	Accept
TC4	50 kg	Accept
TC5	100 kg	Accept
TC6	100.01 kg	Reject

Robust BVA tests values below and above the valid range in addition to values at the boundaries.

C. Additional input combinations

Input	Valid values	Invalid values
Weight	$0<w\leq100$ $0<w\leq100$ kg	Zero, negative, above 100 kg
Destination	Supported zones	Unknown zone
Delivery speed	Standard, Express	Unsupported option

Test boundary weights with different destination zones and delivery speeds because shipping-cost errors may result from interactions between these fields.

D. Improved testing strategy

- Apply ECP to valid and invalid input classes.
- Apply robust BVA to minimum and maximum weight limits.
- Use decision tables for zone and delivery-speed combinations.
- Verify that invalid inputs never generate misleading shipping costs.
- Check decimal precision, unit conversion, and error messages.

E. Conclusion

Negative testing is essential because real users and external systems may supply invalid values. Robust BVA detects missing validation, prevents incorrect charges, and improves the reliability of shipping-cost calculations.

15. Medical Imaging – Basis Path Testing and Prime-Sized Datasets

A. Problem analysis

The medical imaging software produces inconsistent results when dataset sizes are prime numbers. Random testing may miss this defect because it does not systematically select prime-sized datasets or exercise the code paths associated with them.

Possible causes include incorrect array partitioning, parallel-thread scheduling, uneven workload distribution, or errors in handling the final unprocessed elements.

B. Apply white-box testing

Construct a CFG for dataset validation, parallel processing, result aggregation, and error handling. Identify all decision points and independent execution paths.

Cyclomatic complexity can be calculated using:

$$V(G)=E-N+2P$$

$$V(G)=E-N+2P$$

For a connected graph,

$$V(G)=E-N+2$$

$V(G)=E-N+2$. The resulting value estimates the number of independent basis paths to test.

C. Basis path test cases

Assume the program divides the dataset into chunks and processes any remaining elements.

Test case	Dataset size	Purpose
TC1	0	Empty dataset handling
TC2	1	Minimum valid dataset
TC3	8	Evenly divisible workload

TC4	7	Prime-sized dataset
TC5	11	Different prime-sized dataset
TC6	10	Dataset with a remainder
TC7	Large prime	Parallel processing and resource stability

These values are representative examples. The actual suite should reflect the algorithm's chunk size, array limits, and minimum supported dataset.

D. Improved testing strategy

- Use basis path testing to exercise independent control-flow paths.
- Test loop boundaries, chunk allocation, and remainder handling.
- Compare parallel-processing results against a trusted sequential implementation.
- Repeat tests under different thread counts and execution schedules.
- Use assertions for numerical accuracy, deterministic output where required, memory safety, and completion without deadlocks.

Random testing may still be used as supplementary testing, but it should not replace structured tests for prime sizes and workload boundaries.

E. Conclusion

Basis path testing systematically covers independent execution paths, while carefully selected prime and non-prime dataset sizes expose workload-partitioning defects. Combining structural testing with numerical comparisons and parallel stress testing improves the accuracy, reliability, and stability of medical imaging software.